

Predict Market Regimes before Price Prediction

A Capstone Project Architecture & Implementation Guide

Regime Detection → Regime-Conditioned Transformer Forecasting → Trading Strategy

Design document · 3-week execution plan

Table of Contents

1. Project Framing
2. Data
3. System Architecture
4. Feature Engineering
5. Stage 2 — Regime Detection
6. Stage 3 — Transformer Forecaster
7. Stage 4 — Trading Strategy
8. Stage 5 — Evaluation & Robustness
9. Tech Stack & Repository Layout
10. Leakage Checklist
11. Three-Week Timeline
12. What Elevates This From Good to Excellent
13. Reading List

1. Project Framing

Thesis. Financial returns are non-stationary. A single global forecaster averages over incompatible dynamics — trending, mean-reverting, and crisis regimes each have different conditional distributions. Detect the latent regime first, then forecast *conditional on regime*.

Testable claim. Regime-conditioned forecasting beats an identical unconditional model on Sharpe and directional accuracy. This gives a clean ablation, which is the core of a good capstone: one hypothesis, one control, one measured effect size.

Why this framing matters for grading. Many student projects are “I built a Transformer and it made money.” That is unfalsifiable and usually leaked. Your project instead states a hypothesis, builds the control that isolates it, and reports the effect with confidence intervals. If regime-conditioning turns out *not* to help, that is still a publishable finding — and examiners reward the honesty far more than a suspiciously perfect backtest.

2. Data

2.1 Universe

Option	Detail	Notes
Primary	SPY / S&P 500, daily , 2000–2024 (yfinance)	Spans dot-com bust, GFC, COVID, 2022 bear. You <i>need</i> multiple crisis regimes or regime detection is unfalsifiable.
Secondary	BTC-USD, hourly , 2018–2024 (Binance API)	More data, cleaner regimes, no market-hours gaps. Use as a robustness check.

Recommendation: SPY daily as primary, BTC hourly as robustness.

2.2 Exogenous Features

This table is what separates a serious project from a toy. Price-only regime detection finds volatility clusters and nothing else.

Source	Series	Regime signal
FRED (<code>pandas-datareader</code>)	VIX, VIX9D/VIX3M ratio	Volatility regime, term structure of fear
FRED	10Y-2Y term spread (<code>T10Y2Y</code>)	Macro / recession regime
FRED	BAA-AAA credit spread	Corporate stress
FRED	TED spread / SOFR-OIS	Funding liquidity
yfinance	TLT, GLD, DXY, HYG/LQD ratio	Cross-asset risk-on / risk-off
Market data	Advance-decline line, put-call ratio	Breadth, sentiment

Point-in-time (PIT) alignment. FRED macro series are *revised*. The value of a series “as of” a historical date is not the value you download today. If you want full rigour, pull ALFRED vintages and use the *release date*, not the reference date. At minimum, lag all macro series by their typical publication delay (e.g. 1 month for GDP-linked series) and document that you did so.

2.3 Splitting — Non-Negotiable

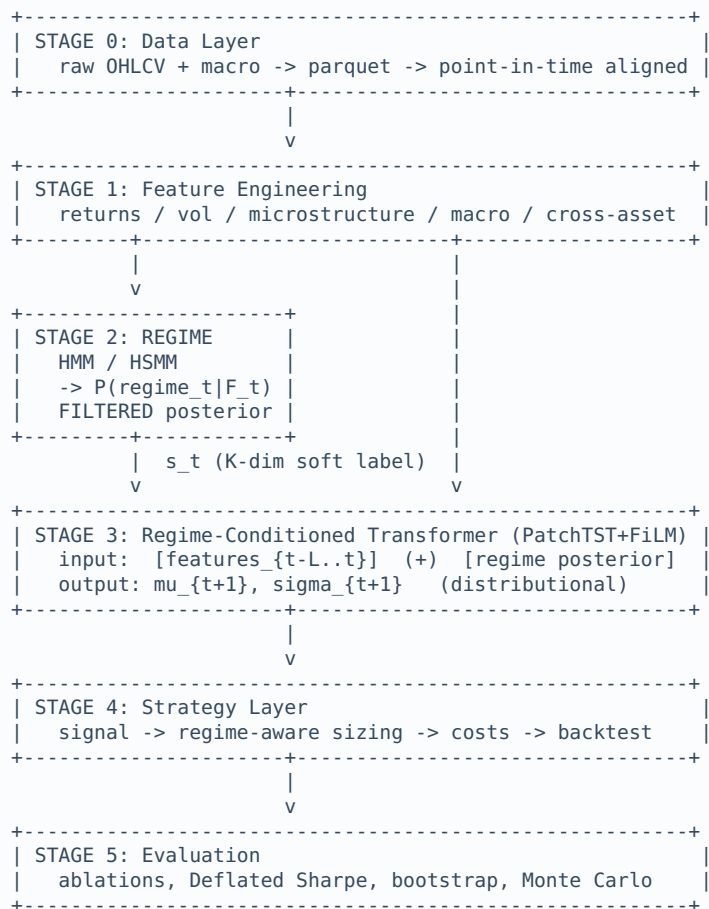
Purged walk-forward CV with embargo (Lopez de Prado). Never random K-fold. Never shuffle.

```
Train [2000-2010] -> Embargo(20d) -> Test [2010-2012]
Train [2000-2012] -> Embargo(20d) -> Test [2012-2014]
Train [2000-2014] -> Embargo(20d) -> Test [2014-2016]
... expanding window ...

Final holdout: 2022-2024. Touched exactly ONCE, on Day 19.
```

The **purge** removes training samples whose label horizon overlaps the test set. The **embargo** removes a buffer after the test set to kill serial-correlation leakage. Both are required.

3. System Architecture



The critical interface is the arrow from Stage 2 to Stage 3: a **K-dimensional filtered posterior** $s_t = [P(\text{regime}=1 \mid F_t), \dots, P(\text{regime}=K \mid F_t)]$. Everything downstream depends on this vector being causal. See Section 5.2.

4. Feature Engineering

Golden rule. Every feature at time t uses only data available *at or before* t . Compute rolling statistics with `.rolling()` then `.shift(1)`. Test this mechanically.

4.1 Regime Features (slow-moving, feed the HMM)

Keep this set **small** — 4 to 6 features. HMMs degrade badly in high dimensions; the full-covariance parameter count grows as $K \cdot d(d+1)/2$.

```

r_t                # log return
realized_vol_20d   # sqrt(252) * rolling std of daily log returns
vol_of_vol_60d     # rolling std of realized_vol (regime instability)
skew_60d, kurt_60d # higher moments -- separates crash from grind-down
trend_strength     # (SMA20 - SMA100) / realized_vol [vol-normalised]
term_spread_z      # z-scored T10Y2Y
credit_spread_z    # z-scored BAA - AAA

```

4.2 Forecasting Features (fast-moving, feed the Transformer)

Group	Features
Returns	<code>r_{t-1..t-L}</code> ; overnight vs. intraday return split
Volatility	Parkinson, Garman-Klass, Yang-Zhang estimators (strictly better than close-to-close: they use the full OHLC bar)
Momentum	12-1 momentum, RSI, MACD histogram — all vol-normalised
Microstructure	Amihud illiquidity, volume z-score, Roll spread estimate
Cross-asset	60d rolling corr(TLT, SPY), DXY return, HYG-LQD spread
Calendar	Day-of-week, month, days-to-FOMC — sin/cos encoded
Regime	<code>[p(s=1), ..., p(s=K)]</code> ← <i>the whole point of the project</i>

4.3 Preprocessing

Fractional differencing ($d \approx 0.3-0.5$) instead of naive first differences on price. First-differencing achieves stationarity but destroys all memory; fractional differencing achieves stationarity while *preserving* long-memory structure. Roughly 30 lines to implement yourself, and doing so signals you know the literature.

```

import numpy as np

def frac_diff_weights(d: float, thresh: float = 1e-5) -> np.ndarray:
    """Binomial expansion weights for fractional differencing."""
    w = [1.0]
    k = 1
    while True:
        w_k = -w[-1] * (d - k + 1) / k
        if abs(w_k) < thresh:
            break
        w.append(w_k)
        k += 1
    return np.array(w[::-1]) # oldest weight first

def frac_diff(series: np.ndarray, d: float, thresh: float = 1e-5) -> np.ndarray:
    """Fixed-width-window fractional differencing (Lopez de Prado, Ch. 5)."""
    w = frac_diff_weights(d, thresh)
    width = len(w) - 1
    out = np.full(len(series), np.nan)
    for i in range(width, len(series)):
        out[i] = np.dot(w, series[i - width : i + 1])
    return out

```

Choose `d` as the **smallest** value passing an ADF stationarity test — that maximises retained memory.

Other preprocessing:

- **Scaling:** expanding-window `StandardScaler`, fit on train only, **refit each walk-forward fold**.
- **Winsorise** at the 1st / 99th percentile (fat tails will otherwise dominate gradients).

4.4 Target

Use **volatility-normalised forward return**:

```
y_t = r_{t+1} / sigma_hat_t
```

Predicting raw returns is dominated by volatility clustering — the model just learns to predict volatility and you get a great-looking R^2 with zero trading edge.

Secondary target (classification variant): triple-barrier labelling (Lopez de Prado) — profit-take barrier, stop-loss barrier, and a vertical time barrier. Label is whichever is touched first. This is a far more honest formulation of “did the trade work” than sign-of-next-return.

5. Stage 2 — Regime Detection

5.1 Primary Model: Gaussian HMM

```
hmmlearn.GaussianHMM, covariance_type="full".
```

Why HMM over clustering: an HMM models *transition dynamics*. It gives calibrated posteriors and its Viterbi decoding is temporally coherent. K-means gives you no persistence — it will flip regimes every other day and your “regime” feature becomes noise.

5.2 The Single Most Important Correctness Issue

Use FILTERING (forward algorithm), not SMOOTHING (forward-backward).

`hmmlearn`'s `predict_proba()` returns **smoothed** posteriors $P(s_t | x_{1..x_T})$ — conditioned on the *entire* series, including the future. Feeding these downstream leaks future information and produces a beautiful, worthless backtest.

You need **filtered** posteriors $P(s_t | x_{1..x_t})$ — conditioned only on the past. This means implementing the forward pass yourself.

```
import numpy as np
from scipy.special import logsumexp

def filtered_posteriors(model, X: np.ndarray) -> np.ndarray:
    """
    Causal  $P(s_t | x_{1..x_t})$  via the forward algorithm.

    NOT hmmlearn's predict_proba(), which runs forward-backward and
    returns SMOOTHED posteriors  $P(s_t | x_{1..x_T})$  -- these use future
    data and will leak.

    Returns
    -----
    (T, K) array of filtered state probabilities.
    """
    framelogprob = model._compute_log_likelihood(X)  # (T, K)
    T, K = framelogprob.shape

    log_alpha = np.zeros((T, K))
    log_A = np.log(model.transmat_ + 1e-300)        # (K, K)
    log_pi = np.log(model.startprob_ + 1e-300)      # (K,)

    log_alpha[0] = log_pi + framelogprob[0]
    for t in range(1, T):
        log_alpha[t] = (
            logsumexp(log_alpha[t - 1][:, None] + log_A, axis=0)
            + framelogprob[t]
        )

    # normalise row-wise in log space -> probabilities
    return np.exp(log_alpha - logsumexp(log_alpha, axis=1, keepdims=True))
```

Second leakage trap: refit the HMM **only on the training window of each walk-forward fold**, then filter *forward* through the test window using those frozen parameters. Do not fit the HMM once on the full history — the EM algorithm will have seen your test set.

```

for train_idx, test_idx in purged_walk_forward(dates, embargo=20):
    hmm = GaussianHMM(n_components=K, covariance_type="full",
                      n_iter=200, random_state=0)
    hmm.fit(X_regime[train_idx])           # params from TRAIN only

    # filter forward through train+test with FROZEN params
    s_full = filtered_posteriors(hmm, X_regime[: test_idx[-1] + 1])
    s_train, s_test = s_full[train_idx], s_full[test_idx]

```

State-label permutation. EM does not preserve state ordering across refits — regime 0 in fold 3 may be regime 2 in fold 4. Canonicalise after every fit: sort states by e.g. mean realized volatility, then permute `means_`, `covars_`, `transmat_`, and `startprob_` consistently. Otherwise your regime feature is scrambled between folds and the downstream model learns nothing.

5.3 Choosing K

Fit `K = 2..6`. Select by three criteria jointly:

1. **BIC** — penalised likelihood.
2. **Regime persistence** — mean state duration should be ≥ 10 days. If states last 2 days, K is too high and you are fitting noise.
3. **Economic interpretability** — can you name the states?

Expect **K = 3 or 4** to win, typically resolving to something like:

State	Character
1	Low-volatility bull / grind-up
2	High-volatility bear
3	Transition / choppy sideways
4	Crisis (rare, extreme vol, extreme negative skew)

5.4 Alternative Regime Models (comparison section)

This is where you earn marks — a single HMM is a B; a comparative study is an A.

Model	Library	Why include it
HSMM (Hidden Semi-Markov)	<code>pyhsmm</code>	Explicit duration modelling . Fixes the HMM's implicit geometric-duration assumption — real market regimes are <i>not</i> geometrically distributed. This is a genuine, well-motivated improvement, not a gimmick.
Markov-Switching AR	<code>statsmodels.MarkovRegression</code>	Hamilton's classic (1989). Switching mean <i>and</i> variance. Econometrically respectable, and a strong baseline that grounds your work in the finance literature.
Change-point detection (PELT)	<code>ruptures</code>	Non-parametric sanity check on regime <i>boundaries</i> . Do the HMM's transitions line up with detected change points?
Wasserstein k-means	custom	Cluster on rolling <i>return distributions</i> rather than point features, using Wasserstein distance. Novel, cheap (~50 lines), makes a great "extension" section.
GMM-VAE latent clustering	<code>torch</code>	Optional deep alternative. Only if time permits.

Timebox `pyhsmm`. It has awkward dependencies and can eat a day. Give it 4 hours; if it fights you, fall back to Markov-Switching AR as your duration-aware comparison and note the substitution in the report.

5.5 Validating Regimes

Regimes are **unsupervised** — you must justify that they are real and not an artefact of the algorithm.

1. **Per-regime summary table:** mean return, annualised volatility, Sharpe, max drawdown, mean duration, and the full transition matrix.

2. **Viterbi overlay on the price chart.** Do the high-volatility states line up with Oct 2008, Mar 2020, and 2022? If not, your regimes are meaningless.
3. **Exogenous ground truth.** Cross-check against NBER recession dates and $VIX > 30$ episodes. Report a confusion matrix or Cohen's kappa.
4. **Stability.** Refit on a window shifted by 6 months. Does the regime path change materially? A regime model that is not stable to reasonable perturbation is not a regime model.

6. Stage 3 — Transformer Forecaster

Do NOT use a vanilla NLP Transformer. The recent time-series literature is blunt: on many benchmarks, a *single linear layer* (DLinear, Zeng et al. 2022) beats naive Transformers. You must use a time-series-aware variant, and you must beat that linear baseline. If you cannot beat DLinear, you do not have a result.

6.1 Backbone: PatchTST

Why PatchTST (Nie et al. 2023):

- **Patching** — segment the series into subseries patches. Reduces attention's sequence length quadratically and lets each token carry *local semantics* rather than a single meaningless timestep.
- **Channel independence** — each feature series is processed independently, which regularises heavily and works well on noisy data.
- Strong published performance, well-cited, easy to implement from scratch.

6.2 Architecture

```
Input window L = 60 days, F features
|
+-- Patching: patch_len = 16, stride = 8 -> N patches
+-- Linear embedding -> d_model = 128
+-- Learnable positional encoding
|
+-- REGIME CONDITIONING (the novel component):
|
|     gamma, beta = MLP( s_t )           # s_t = filtered posterior, K-dim
|     h <- gamma * LayerNorm(h) + beta   # FiLM, applied per encoder block
|
|     (alternative: cross-attention from h to a learned
|     regime-embedding table of size K x d_model)
|
+-- 3 x Encoder blocks:
|     MultiHeadSelfAttention(heads = 4)
|     -> FeedForward
|     -> pre-LayerNorm, dropout = 0.2
|
+-- Head -> (mu, log_sigma)           # distributional output
```

6.3 Why FiLM Rather Than Concatenation

Concatenating the regime vector to the input lets the model **ignore** it — it is 3 features among 40, and gradient descent has no reason to attend to it.

FiLM (Feature-wise Linear Modulation, Perez et al. 2018) *forces* regime information to modulate every layer's activations: the regime controls the affine transform applied to every hidden unit. It cannot be ignored.

This is a defensible architectural decision you can articulate in a viva, which is exactly what a capstone examiner is probing for.

```
import torch
import torch.nn as nn

class FiLM(nn.Module):
    """Feature-wise Linear Modulation conditioned on the regime posterior."""

    def __init__(self, n_regimes: int, d_model: int, hidden: int = 64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_regimes, hidden),
            nn.ReLU(),
            nn.Linear(hidden, 2 * d_model),
        )
        self.norm = nn.LayerNorm(d_model)

    def forward(self, h: torch.Tensor, s: torch.Tensor) -> torch.Tensor:
        """
        h : (B, N, d_model)    hidden states
        s : (B, K)              filtered regime posterior at time t
        """
        gamma, beta = self.net(s).chunk(2, dim=-1)  # each (B, d_model)
        gamma = gamma.unsqueeze(1)                  # (B, 1, d_model)
        beta = beta.unsqueeze(1)
        return gamma * self.norm(h) + beta
```

Initialise the final layer of `self.net` to zeros so that `gamma = 0`, `beta = 0` at start — then add 1 to gamma. The block begins as an identity map and *learns* to deviate. This materially stabilises early training.

6.4 Loss: Distributional, Not Point

Gaussian negative log-likelihood, so the model outputs its own uncertainty:

```
L = 0.5 * ( log(sigma^2) + (y - mu)^2 / sigma^2 )
```

```
def gaussian_nll(mu, log_sigma, y, log_sigma_min=-5.0, log_sigma_max=2.0):
    log_sigma = log_sigma.clamp(log_sigma_min, log_sigma_max)  # stability
    sigma_sq = torch.exp(2.0 * log_sigma)
    return 0.5 * (2.0 * log_sigma + (y - mu) ** 2 / sigma_sq).mean()
```

Uncertainty is not a nice-to-have. You will use `sigma` directly for position sizing in Stage 4. This is what connects the forecasting layer to the strategy layer in a principled way, rather than bolting a threshold onto a point forecast.

Alternative: quantile (pinball) loss at [0.1, 0.5, 0.9], and size on the interquantile spread. Equally defensible; pick one.

6.5 Regularisation

Financial data has terrible signal-to-noise. **Regularisation matters more than architecture here.**

- Dropout 0.2-0.3; weight decay 1e-4.
- **Early-stop on validation Sharpe, not MSE.** Low MSE does not imply profitability — a model that predicts 0 everywhere has excellent MSE and zero edge.
- **Keep the model small.** `d_model = 128`, 3 layers. Anything larger memorises noise.
- **Ensemble 5 seeds**, average predictions. A single-seed result on financial data is noise; reviewers know this.

6.6 Baselines You MUST Beat

The ablation table is the **heart of your evaluation**. Build it before you build the Transformer.

#	Model	Purpose
0	Buy & hold	Did you beat doing nothing?
1	Random walk / zero forecast	Is there <i>any</i> signal?
2	ARIMA-GARCH (<code>arch</code>)	Classical econometrics benchmark
3	DLinear	The linear model that embarrasses Transformers
4	LightGBM on identical features	Trees usually win on tabular finance
5	Transformer, NO regime input	← the key control
6	Transformer + FiLM regime	← the thesis
7	K separate Transformers, hard switching	Soft vs. hard conditioning

If model #6 does not beat model #5, that is a FINDING, not a failure. Report it honestly and diagnose *why*: are the regimes unstable out-of-sample? Is the posterior near-uniform (i.e. the HMM is uncertain most of the time)? Is the effect swamped by transaction costs? Examiners reward this analysis enormously more than a suspiciously clean positive result they suspect is leaked.

7. Stage 4 — Trading Strategy

Forecasting accuracy is **not** profit. This is where student projects usually collapse.

7.1 Signal to Position

```
import numpy as np

# Kelly-inspired, uncertainty-aware, regime-gated sizing
edge      = mu_hat / (sigma_hat ** 2)          # Kelly-lite
raw_pos    = np.clip(edge, -1.0, 1.0)

vol_target = 0.10                             # 10% annualised
scaling    = vol_target / (realized_vol_20d * np.sqrt(252))

position   = raw_pos * scaling * regime_gate[np.argmax(s_t)]
position    = np.clip(position, -1.0, 1.0)      # leverage cap
```

7.2 Regime Gating

Learn the gates **on the training fold only**. Never hand-tune them by looking at test performance — that is the most common form of subtle overfitting in this kind of project.

Regime	Gate
Low-vol bull	Full size, long bias
High-vol bear	Reduced size, or flat; enable shorts if the mandate permits
Crisis	Flat. Capital preservation.
Transition / choppy	Half size

Learning to sit out is a legitimate and valuable result. A strategy that goes flat through 2008 and March 2020 and simply avoids the drawdown will beat a heroic one on Calmar. Do not treat “the model does nothing in crisis” as a failure to be tuned away.

7.3 Frictions — Omit These and the Backtest Is Fiction

Friction	Model
Commission	1-5 bps per side
Slippage	Half-spread + market impact $\approx k * (\text{volume} / \text{ADV})^{0.5}$
Turnover penalty	Add <code>lambda * abs(delta_position)</code> to the objective, or you will trade every single bar
Execution timing	Trade at the next bar’s open, never the signal bar’s close
Short borrow	Include a borrow cost if you allow shorts

7.4 Metrics

- Sharpe, **Sortino**, Calmar
- Max drawdown **and drawdown duration** (duration is what actually gets a fund shut down)
- Turnover; report **gross AND net** returns so cost drag is visible
- Hit rate, profit factor, average win / average loss
- **Per-regime P&L attribution** ← *this is what demonstrates the regime layer is doing real work*
- **Deflated Sharpe Ratio** (Bailey & Lopez de Prado) — adjusts the Sharpe for the number of configurations you tried. Including this signals genuine methodological maturity.

8. Stage 5 — Evaluation & Robustness

1. **Ablation table** (Section 6.6) — this is your headline result.
2. **Statistical significance.** Stationary bootstrap on the return series (Politis-Romano); block-bootstrap confidence interval on the Sharpe ratio.

A Sharpe of 1.2 with a 95% CI of $[-0.4, 2.8]$ is **not a result**. Report the interval, always.

3. **Monte Carlo.** Shuffle trade order $1,000\times \rightarrow$ drawdown distribution. Where does your realised max-DD sit in it?
4. **Sensitivity.** Vary K , window length L , and transaction costs (0 / 5 / 10 / 20 bps). **Does the edge survive at 10 bps?** If it dies at 5 bps, it was never real.
5. **Regime attribution.** Does the Transformer's *conditional* behaviour actually differ across regimes? Compare per-regime prediction distributions, and inspect the learned FiLM γ / β vectors across regimes. If they are near-identical, FiLM is not learning anything and you should say so.
6. **Interpretability.** Attention maps; SHAP on the LightGBM baseline as a proxy for feature importance; the regime transition matrix.

The single best diagnostic you can run. Shuffle your target labels and rerun the entire pipeline end to end. Sharpe should collapse to approximately zero.

If it does not, you have leakage. Find it before you write a single word of the report.

9. Tech Stack & Repository Layout

9.1 Stack

Layer	Tools
Data	yfinance, pandas-datareader (FRED), pandas, pyarrow
Regime	hmmlearn, statsmodels (MarkovRegression), ruptures, pyhsmm
Model	pytorch, pytorch-lightning
Baselines	lightgbm, arch (GARCH), statsmodels (ARIMA)
Backtest	vectorbt (fast, vectorised, good metrics) + your own event loop as a cross-check
Tracking	mlflow or wandb
Config	hydra — essential once the ablation grid has 8 models × 5 seeds × N folds
Viz	plotly, matplotlib

Run vectorbt and a hand-written event loop side by side on the same signals. If they disagree, one of them has a lookahead bug. This cross-check has saved more backtests than any other single practice.

9.2 Repository Layout

```
project/
+-- conf/                # hydra yaml configs (models, folds, costs)
+-- src/
|   +-- data/            # loaders, PIT alignment
|   +-- features/        # engineering, fracdiff, scalars
|   +-- regimes/
|       |   +-- hmm.py
|       |   +-- hsmm.py
|       |   +-- filtering.py # <-- causal forward algorithm
|       |   +-- validate.py
|   +-- models/
|       |   +-- patchtst.py
|       |   +-- film.py
|       |   +-- dlinear.py
|       |   +-- baselines.py
|   +-- backtest/
|       |   +-- engine.py
|       |   +-- costs.py
|       |   +-- sizing.py
|       |   +-- metrics.py
|   +-- eval/
|       |   +-- ablation.py
|       |   +-- bootstrap.py
|       |   +-- dsr.py
+-- notebooks/           # EDA + final results ONLY -- not the pipeline
+-- tests/               # <-- leakage tests live here
+-- reports/
```

Keep the pipeline in `src/`, not in notebooks. Notebooks hide state, execute out of order, and are where lookahead bias goes to breed. Use them for EDA and final figures only.

10. Leakage Checklist

This is the single most common way capstone finance projects fail. Write each of these as a `pytest` test in `tests/test_leakage.py`.

☐	HMM posteriors are filtered , not smoothed (<code>filtered_posteriors</code> , not <code>predict_proba</code>)
☐	HMM is refit per walk-forward fold , on train data only
☐	HMM state labels are canonicalised after each refit (sorted by volatility)
☐	<code>StandardScaler</code> is <code>fit</code> on train only, refit per fold
☐	All rolling features have <code>.shift(1)</code> applied
☐	Macro data uses the release date , not the reference date
☐	Target is strictly <code>t+1</code> ; no overlapping-window contamination in the loss
☐	Purge + embargo enforced between train and test
☐	Execution happens at the next bar's open
☐	Final holdout (2022–24) is evaluated exactly once
☐	Shuffled-label test passes — Sharpe collapses to ~ 0

11. Three-Week Timeline

Week 1 — Data, Features, Regimes

Day	Task
1	SPY 2000–2024 (<code>yfinance</code>) + BTC hourly (Binance) + FRED macro (VIX, VIX9D/3M, T10Y2Y, BAA-AAA, TED). PIT alignment → parquet. EDA / stylised facts.
2	Feature engineering: returns, Parkinson / Garman-Klass / Yang-Zhang vol, momentum, Amihud, Roll spread, cross-asset (TLT / GLD / DXY / HYG-LQD), calendar. Fractional differencing ($d \approx 0.3-0.5$).
3	Purged walk-forward CV + embargo. Expanding-window scalars. Write <code>tests/test_leakage.py</code> . Triple-barrier labels as secondary target.
4	HMM (<code>hmmlearn</code> , full covariance, 5–6 regime features). $K = 2..6$, select by BIC + persistence + interpretability. Causal forward-filtering (not <code>predict_proba</code>). Refit-per-fold loop.
5	HSMM (<code>pyhsmm</code>), Markov-Switching AR (<code>statsmodels</code>), PELT change-points (<code>ruptures</code>), Wasserstein k-means.
6	Regime validation: per-regime stats, transition matrix, Viterbi overlay, NBER / VIX cross-check, refit stability.
7	Hydra configs + MLflow tracking. Buffer.

Week 2 — Models

Day	Task
8	Baselines: buy & hold, random walk, ARIMA-GARCH (<code>arch</code>), DLinear , LightGBM .
9	PatchTST: patching (16 / 8), embedding, positional encoding, 3× encoder blocks, <code>d_model = 128</code> .
10	Gaussian NLL head + quantile (pinball) variant. Train the no-regime control (#5) . Early-stop on validation Sharpe.
11	FiLM conditioning: <code>gamma</code> , <code>beta = MLP(filtered posterior)</code> , per-block modulation. Cross-attention variant as the alternative.
12	Train the regime-conditioned model (#6). 5-seed ensemble.
13	Hard-switching variant: K separate Transformers (#7).
14	Full ablation grid (models 0–7) across all folds. Buffer.

Week 3 — Strategy, Evaluation, Writeup

Day	Task
15	Backtest engine (<code>vectorbt</code> + own event loop cross-check). Next-open execution, commission 1-5 bps, $k \cdot (\text{vol}/\text{ADV})^{0.5}$ impact, turnover penalty, borrow cost.
16	Sizing: μ / σ^2 Kelly-lite, 10% vol target, regime gating (learned on train only), leverage cap.
17	Metrics: Sharpe, Sortino, Calmar, MaxDD + duration, turnover, gross/net drag, hit rate, profit factor, per-regime P&L attribution .
18	Stationary bootstrap (Politis-Romano) CI on Sharpe. Monte Carlo trade-order shuffle → DD distribution. Sensitivity: K , L , costs (0 / 5 / 10 / 20 bps).
19	Deflated Sharpe Ratio . Shuffled-label sanity check. Interpretability: attention maps, SHAP on LGBM, FiLM γ / β across regimes. BTC robustness check. Final holdout (2022-24) — evaluated ONCE .
20	Figures + report writeup.
21	Slides, polish, buffer.

Execution Notes

Critical path: Days 3-4. If the purged CV harness or the causal filtering slips, everything downstream is contaminated and unrecoverable. Protect these two days above all else — cut regime-validation depth or a baseline model instead.

Parallelise. Day 5 (alternative regime models) and Day 8 (baselines) are independent of the main line. Run them in the background / overnight while you build the Transformer.

Watch. Days 5 and 13 are the tightest. If `pyhsmm` fights you, timebox it to 4 hours and fall back to Markov-Switching AR as the duration-aware comparison.

If you fall behind, cut in this order:

1. Model #7 (hard switching)
2. GMM-VAE / Wasserstein k-means
3. K -sensitivity analysis
4. Seed ensembling (drop to 3 seeds, not 5)
5. BTC robustness check

Never cut: causal filtering · purged CV · transaction costs · **model #5 (the no-regime control)**.

Without #5 you have no thesis test, and without a thesis test you have no capstone.

12. What Elevates This From Good to Excellent

1. **Causal filtering of HMM posteriors.** Most projects get this wrong and do not know it. Getting it right — and *explaining why it matters* — is an immediate differentiator.
2. **FiLM conditioning rather than concatenation.** A real architectural argument you can defend under questioning.
3. **Distributional output → uncertainty-aware sizing.** Connects the forecasting layer to the strategy layer properly, rather than bolting a threshold onto a point forecast.
4. **Deflated Sharpe Ratio.** Demonstrates you understand backtest overfitting and multiple-testing bias.
5. **Honest negative results.** If regime conditioning does not help, say so and diagnose why.
6. **HSMM vs. HMM comparison.** Duration modelling is a genuine, well-motivated improvement grounded in the structure of the problem.

The Narrative That Sells This

“I hypothesised that regime-conditioning improves forecasting. Here is a rigorously leak-free pipeline. Here is the ablation that isolates the regime contribution. Here is the effect size with confidence intervals. Here is where it works and where it does not.”

That is a capstone.

13. Reading List

Topic	Reference
Financial ML methodology, purging, embargo, fractional differencing, triple-barrier, DSR	Lopez de Prado, <i>Advances in Financial Machine Learning</i> (2018) — read Chapters 2, 3, 5, 7 before Day 1
Deflated Sharpe Ratio	Bailey & Lopez de Prado, “The Deflated Sharpe Ratio” (2014)
Regime switching in econometrics	Hamilton, “A New Approach to the Economic Analysis of Nonstationary Time Series” (1989)
PatchTST	Nie et al., “A Time Series is Worth 64 Words” (ICLR 2023)
Linear baselines beating Transformers	Zeng et al., “Are Transformers Effective for Time Series Forecasting?” (AAAI 2023)
FiLM conditioning	Perez et al., “FiLM: Visual Reasoning with a General Conditioning Layer” (AAAI 2018)
Stationary bootstrap	Politis & Romano, “The Stationary Bootstrap” (1994)
Volatility estimators	Yang & Zhang, “Drift-Independent Volatility Estimation” (2000)